



Verwaltungsassistent Sicherheitsarchitektur

Sicherheits- und Deployment-Architektur des
Enterprise Knowledge Reasoning Platform (Verwaltungsassistent) / Verwaltungsassistent

Version: 1.1 (Security Hardening)

Datum: 16. August 2026

Status: Kundenorientiertes Sicherheitsdokument

Zielgruppe: IT-Verantwortliche, Administratoren, Datenschutzbeauftragte, Sicherheitsprüfer

Basis: Quellcode, Konfiguration und verifizierte fokussierte Tests (Stand 16.08.2026)

Inhalt

Hinweis zur Dokumentation

1. Kurzfassung
 2. Sicherheitsmodell auf einen Blick
 3. Architektur
 4. Client-Server-Kommunikation
 5. Anmeldung und Authentifizierung
 6. Tokens und Sitzungen
 7. Benutzerrechte und Rollen
 8. Administration
 9. Audit und Nachvollziehbarkeit
 10. Browser- und API-Sicherheit
 11. Dokumentensicherheit und Verschlüsselung at Rest
 12. Soll Verschlüsselung at Rest eingeführt werden?
 13. AI-/LLM-Datenschutz
 14. Air-Gap / Offline-Betrieb
 15. Datenflüsse
 16. Bedrohungsmodell
 17. Produktionsdeployment (Empfehlung für eine Kommune)
 18. Verantwortungsgrenzen — was Verwaltungsassistent nicht leistet
 19. Sicherheitsstatus
 20. Empfehlungen (priorisiert)
 21. FAQ
- Anhang A — TLS-Konfiguration (Reverse Proxy, Beispiel)

Verwaltungsassistent Sicherheitsarchitektur

Sicherheits- und Deployment-Architektur des Enterprise Knowledge Reasoning Platform (Verwaltungsassistent) / Verwaltungsassistent

Version 1.0 · 16. August 2026 · Zielgruppe: IT-Verantwortliche, Administratoren, Datenschutzbeauftragte, Sicherheitsprüfer

Hinweis zur Dokumentation

Dieses Dokument beschreibt, **was aktuell implementiert ist**, getrennt von **Empfehlungen für den Produktivbetrieb**. Drei Kennzeichnungen werden durchgängig verwendet:

- **AKTUELL IMPLEMENTIERT** — im Quellcode vorhanden und (wo möglich) durch Tests zur Laufzeit verifiziert.
- **EMPFOHLENE PRODUKTIONSKONFIGURATION** — vom Deployment/Kunden umzusetzen.
- **EMPFOHLENE WEITERENTWICKLUNG** — von Verwaltungsassistent zu implementieren.

Alle Aussagen wurden am 16.08.2026 gegen den aktuellen Quellcode und die laufende Demo-Installation geprüft (15 fokussierte Sicherheitstests, alle bestanden; zusätzlich 171 Tests der schnellen Suite). Wo eine Lücke besteht, wird sie ausdrücklich benannt.

1. Kurzfassung

Verwaltungsassistent ist eine lokal betreibbare Frage-Antwort- und Entscheidungsplattform für öffentliche Verwaltungen. Die Referenzinstallation („Verwaltungsassistent“) läuft vollständig auf eigener Infrastruktur: Browser-Anwendung, PostgreSQL, Qdrant (Vektorsuche), Neo4j (Wissensgraph) und ein lokaler Modellserver (Ollama) für die KI-Verarbeitung.

Wichtigste Sicherheitsmerkmale (aktuell implementiert):

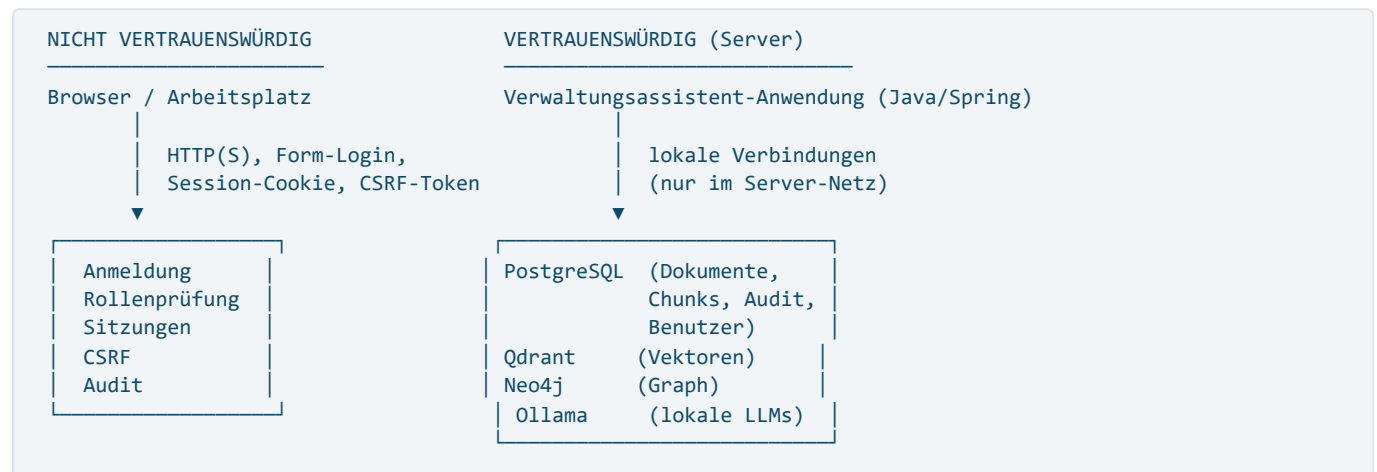
- Passwörter werden als BCrypt-Hash (Kostenfaktor 12) gespeichert — niemals im Klartext.
- Anmeldung über Formular-Login mit serverseitigen Sitzungen; CSRF-Schutz aktiv (verifiziert: POST ohne Token → 403).
- Rollenbasierte Zugriffskontrolle über die Security-Filterkette: ADMIN, ANALYST, USER (plus AUDITOR für Audit/Korpus).
- Benutzerverwaltung mit Sperr-/Lösch-Schutz: kein Selbst-Sperren, kein Entfernen des letzten Administrators; gesperrte Benutzer werden sofort ausgeloggt.
- Audit-Protokoll für Anmeldungen, Abmeldungen, Benutzerverwaltung, Dokumente, Abfragen und KI-Ausführungen — inkl. IP-Adresse.
- Keine Daten verlassen den Server: Modelle laufen lokal; keine Telemetrie; kein Cloud-Zwang.
- Die KI-Pipeline verweigert Antworten, wenn die Belege unzureichend sind (Fail-Closed) — wichtig gegen plausibel klingende, aber unbelegte Antworten.

Wichtigste bekannte Grenzen (aktueller Stand):

- Dokumente liegen **unverschlüsselt** auf dem Dateisystem und in den Datenbanken (keine Verschlüsselung at rest).
- Die Demo läuft über **HTTP** (kein TLS); HTTPS ist als Produktionskonfiguration über einen Reverse Proxy vorgesehen.
- Kein Schutz gegen Brute-Force-Angriffe auf das Login (keine Ratenbegrenzung, keine automatische Sperre nach Fehlversuchen).
- Fehlgeschlagene Anmeldungen werden zwar protokolliert, das Audit-Ereignis geht aber durch einen Transaktions-Rollback verloren (verifizierte Lücke, Abschnitt 9).
- Die Fall-Detailansicht prüft keine Eigentümerschaft anhand der Fall-ID (UUID); die Liste filtert nach Eigentümer. Das System ist für eine Organisation gedacht, nicht für mandantenfremde Benutzer.

Diese Punkte werden in Abschnitt 20 (Empfehlungen) priorisiert behandelt.

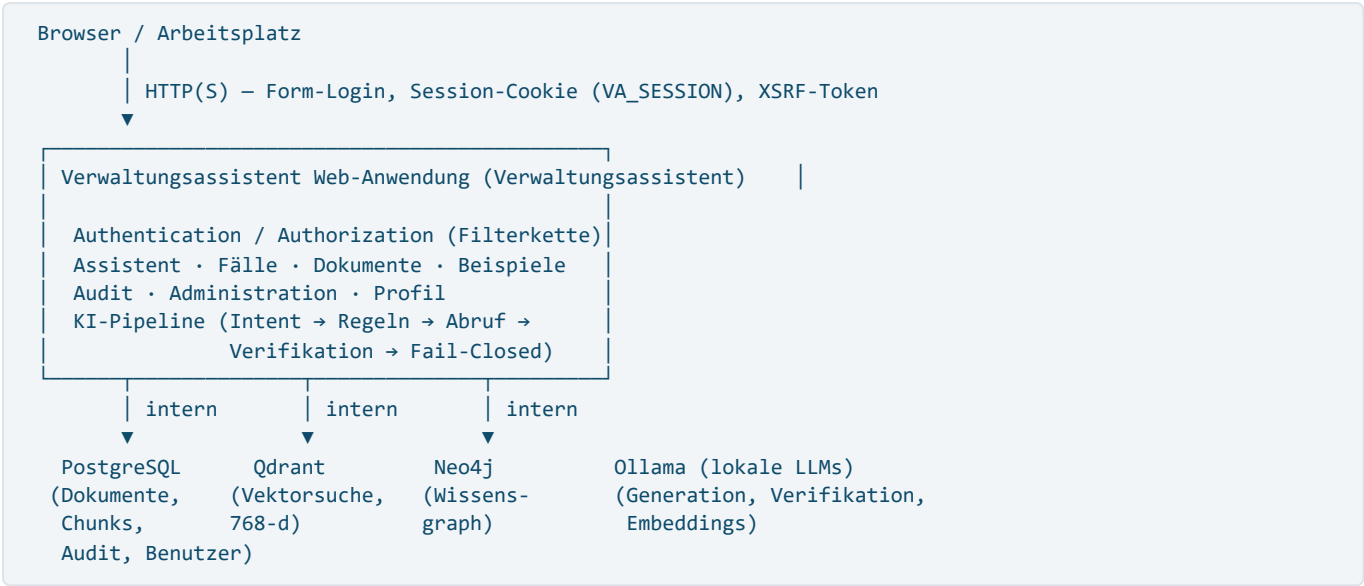
2. Sicherheitsmodell auf einen Blick



Grundprinzipien:

1. **Nichts ist öffentlich.** Alle fachlichen Seiten erfordern eine Anmeldung; öffentlich sind nur Login-Seite und statische Ressourcen.
2. **Rechte werden an der Filterkette erzwungen**, nicht nur im UI versteckt (verifiziert durch Rollen-Tests).
3. **Der Server vertraut dem Browser nicht blind:** CSRF-Token für alle zustandsändernden Formulare.
4. **Alles Relevante wird protokolliert:** Anmeldung, Abmeldung, Verwaltungsaktionen, KI-Ausführungen — mit Zeitstempel, Benutzer und IP.
5. **Lokale Verarbeitung:** Fragen, Dokumente und KI-Aufrufe verlassen den Server nicht.

3. Architektur



Verbindungen — was ist intern, was kann das Netz verlassen?

Verbindung	Richtung	Status
Browser → Verwaltungsassistent	extern (Behördennetz)	die einzige externe Verbindung; im VM-Deployment der einzige freigegebene Port (8081, zusätzlich SSH für Administration)
Verwaltungsassistent → PostgreSQL / Qdrant / Neo4j	nur Server-intern	Ports 5432/6333/7687 im VM-Deployment nicht von außen erreichbar
Verwaltungsassistent → Ollama	nur Server-intern (localhost:11434)	Modellinferenz lokal
Verwaltungsassistent → Internet	nicht erforderlich im Betrieb	nur Erst-Installation (Modell-Downloads, Container-Images, OS-Updates)

AKTUELL IMPLEMENTIERT: Alle Komponenten laufen auf einem Host (Docker für die Datenbanken, Ollama als lokaler Dienst). Es existiert ein reproduzierbares VM-Bundle (docs/deployer), dessen Firewall nur Web-Port und SSH freigibt.

KI-Anbindung: Der Modellserver (Ollama) ist die aktive Konfiguration. Ein Provider-Modul für OpenAI-kompatible externe APIs existiert im Code, ist aber **nicht aktiv**. Eine Nutzung externer LLM-APIs würde Dokument- und Frageinhalte an den Anbieter senden — das wäre eine bewusste Deployment-Entscheidung, nicht der Standard.

4. Client-Server-Kommunikation

AKTUELL IMPLEMENTIERT: Die Demo-Installation läuft über **HTTP ohne TLS**. Es gibt keine TLS-Terminierung in der Anwendung.

EMPFOHLENE PRODUKTIONSKONFIGURATION:

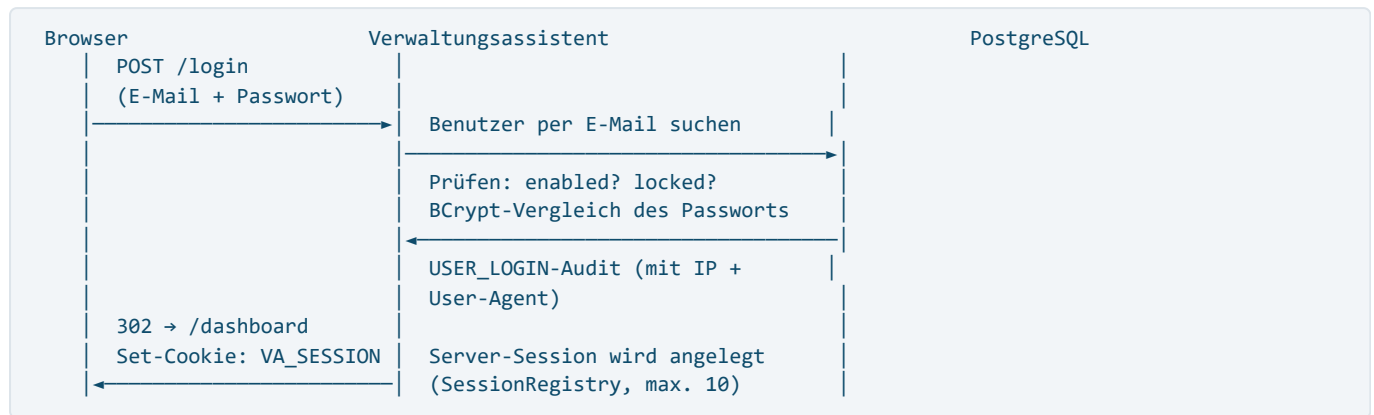
```
Browser — HTTPS → Reverse Proxy (nginx/Caddy o. ä.) — HTTP → Verwaltungsassistent
                        |
                        TLS-Zertifikate,
                        HSTS optional
```

Der Reverse Proxy übernimmt TLS-Terminierung, Zertifikatsverwaltung und optional HSTS. Die interne Verbindung Proxy → Verwaltungsassistent liegt im Server-Netz. Zusätzlich: Session-Cookie-Flags (`secure`, `SameSite`) setzen (Abschnitt 6).

Sitzungscookie: Name `VA_SESSION`, Timeout 30 Minuten (konfigurierbar). Seit dem Härtungs-Update explizit konfiguriert: **HttpOnly=true**, **SameSite=Lax**; `Secure` folgt der Deployment-Konfiguration (`COOKIE_SECURE`, Standard: aus für die lokale HTTP-Demo, **an** hinter dem HTTPS-Reverse-Proxy).

5. Anmeldung und Authentifizierung

Was passiert bei der Anmeldung? (AKTUELL IMPLEMENTIERT, per Test verifiziert)



- **Benutzername** ist die E-Mail-Adresse (normalisiert auf Kleinbuchstaben).
- **Passwörter** werden mit **BCrypt (Kostenfaktor 12)** gehasht gespeichert (Spalte `password_hash`). Es wird nie ein Passwort verschlüsselt oder im Klartext abgelegt — gehasht ist hier das richtige Wort: Ein Hash ist eine Einwegfunktion; selbst mit Datenbankzugriff ist das Passwort nicht rückrechenbar.
- **Fehlgeschlagene Anmeldung:** gleiche Fehlermeldung für „unbekannte E-Mail“ und „falsches Passwort“ (kein User-Enumeration über die Meldung); jeder Fehlversuch wird als `USER_LOGIN_FAILED`-Audit-Ereignis **inkl. IP** persistiert (eigene Transaktion, übersteht den Login-Rollback — per Test verifiziert).
- **Brute-Force-Schutz (AKTUELL IMPLEMENTIERT):** Nach 5 Fehlversuchen innerhalb von 10 Minuten wird die E-Mail-Adresse für 10 Minuten gesperrt (konfigurierbar über `platform.auth.login.*` / Umgebungsvariablen `LOGIN_MAX_FAILURES`, `LOGIN_FAILURE_WINDOW`, `LOGIN_LOCKOUT_DURATION`). Die Sperre ist **rein zeitbasiert** — sie läuft automatisch ab, kein Konto (auch kein Administrator) kann dauerhaft gesperrt werden. Eine erfolgreiche Anmeldung setzt den Zähler zurück. Sperrbedingte Ablehnungen werden mit Grund `too_many_attempts` auditiert (per Test verifiziert).
- **Gesperrte/deaktivierte/gelöschte Benutzer können sich nicht anmelden** (per Test verifiziert).
- **Passwort-Zurücksetzung:** existiert derzeit **nicht** (weder durch den Benutzer noch durch den Administrator). Für den Produktivbetrieb empfohlen: geführter Reset-Prozess (P2).
- **Abmeldung:** Sitzung wird serverseitig invalidiert, Session-Cookie gelöscht, `USER_LOGOUT`-Audit geschrieben (per Test verifiziert).

Registrierung: Die Web-Anwendung hat **keine funktionierende Registrierung** (der Link „Registrieren“ führt ins Leere). Benutzer werden vom Administrator angelegt oder im Demo-Profil automatisch gesetzt. Hinweis: Die separate Plattform-REST-API (nicht Teil der Verwaltungsassistent-Installation) bietet einen öffentlichen Registrierungs-Endpunkt, der Rollen aus der Anfrage übernimmt — **vor einem Einsatz dieser API müssen Registrierung und Rollenvergabe eingeschränkt werden** (kritische Weiterentwicklung, Abschnitt 20).

6. Tokens und Sitzungen

Das System verwendet **zwei verschiedene Modelle**:

Bereich	Modell	Details
Verwaltungsassistent (Browser)	Serverseitige Sitzungen	VA_SESSION-Cookie, 30-min-Timeout, SessionRegistry, max. 10 gleichzeitige Sitzungen pro Benutzer
Plattform-REST-API (separate App im Repository)	JWT + Refresh-Token	HS256-signierte Access-Tokens (15 min), Refresh-Tokens (30 Tage)

Sitzungen (Web, AKTUELL IMPLEMENTIERT):

- Ein aktiver Login = eine HTTP-Session im SessionRegistry. Der Administrator sieht aktive Sitzungen inkl. IP in der Benutzerverwaltung.
- Beim Überschreiten von 10 parallelen Sitzungen wird die **älteste** Sitzung beendet (Spring-Standard).
- Ablauf: `/login?expired` (bzw. HTTP 401 bei htmx-Anfragen, damit die UI sauber weiterleitet).
- Logout invalidiert die Sitzung serverseitig (per Test verifiziert).
- Benutzer-Sperrung beendet alle aktiven Sitzungen des Benutzers sofort (SessionRegistry, Code-verifiziert; Login-Sperre per Test verifiziert).

Tokens (Plattform-API, AKTUELL IMPLEMENTIERT im Code):

- Access-Token: JWT, HS256, Claims `sub` (E-Mail), `user_id`, `roles`; Gültigkeit 15 Minuten.
- Refresh-Token: 64 Byte kryptografischer Zufall; **nur als SHA-256-Hash gespeichert** (kein Klartext-Token in der Datenbank); 30 Tage gültig.
- **Rotation:** Jeder Refresh widerruft den alten Token und erzeugt einen neuen (Verkettung über `replacedByTokenId`; parallele Refreshes werden über optimistisches Sperren abgefangen).
- **Widerruf:** Logout widerruft den Refresh-Token; Benutzer-Löschung entfernt alle Token-Sitzungen des Benutzers.
- Signaturschlüssel: Umgebungsvariable `JWT_SECRET`; **der Standardwert im Code ist ein Entwicklungsschlüssel** („default-dev-jwt-secret-change-in-production“). **Produktionspflicht:** Schlüssel setzen (mind. 32 Byte, erzwungen), geheim halten, rotieren können.

Browser-Speicherung: Die Web-App speichert keine Token im Browser (nur Session-Cookie). Das XSRF-Token-Cookie ist bewusst nicht HttpOnly (die htmx-UI liest es für den CSRF-Header).

7. Benutzerrechte und Rollen

AKTUELL IMPLEMENTIERT: Die Rechteprüfung erfolgt in der **Security-Filterkette** (URL-basiert) — nicht über `@PreAuthorize`-Annotationen auf den Controllern der Web-App (per Quellcode geprüft; die Rollen-Tests bestätigen die Durchsetzung). Es gibt keine zusätzliche Service-Ebenen-Autorisierung.

Rollen: **ADMIN**, **ANALYST**, **USER** (Enum) sowie **AUDITOR** (als Rollenname in der Filterkette).

Funktion	Benutzer (USER)	Analyst	Auditor	Administrator
Anmeldung	✓	✓	✓	✓
Assistent (Fragen)	✓	✓	✓	✓
Fälle (eigene)	✓	✓	✓	✓ (alle)
Entscheidungen	✓	✓	✓	✓
Dokumente	✓	✓	✓	✓
Beispiele / Korpus	✗	✗	✓	✓
Audit-Protokoll	✗	✗	✓	✓
Benutzerverwaltung	✗	✗	✗	✓
Benutzer sperren/entsperren	✗	✗	✗	✓
Benutzer löschen	✗	✗	✗	✓

(Verifiziert per Test: USER → /admin, /audit, /corpus = 403; ADMIN → alles erlaubt; AUDITOR → Audit ja, Admin nein; anonym → überall Weiterleitung auf /login.)

Daten-Sichtbarkeit (aktuelles Modell, Ein-Organisations-Betrieb):

- **Fälle (AKTUELL IMPLEMENTIERT, per Test verifiziert):** Objekt-Ebene-Autorisierung über einen zentralen Zugriffswächter (`CaseAccessGuard`): Administratoren dürfen alle Fälle öffnen; normale Benutzer **nur eigene** Fälle (Eigentümer-E-Mail muss übereinstimmen). Die Prüfung gilt für die Detailansicht **und alle Unteraktionen** (Dokumente, Timeline, Notizen, Checkliste, Phase, Dokument-Verknüpfungen, Entscheidungsanalyse inkl. Fortschritts-Polling, Archivieren). Fremde Fall-IDs → 403; unbekannte IDs → 404; anonym → Weiterleitung zur Anmeldung. Die Liste filtert weiterhin nach Eigentümer (bzw. zeigt Administratoren alle Fälle).
- **Dokumente:** Die Dokumentliste ist für alle angemeldeten Benutzer sichtbar (kein Eigentümer-Filter) — bewusstes Ein-Organisations-Modell. Der Original-Download der Datei ist über die Web-UI derzeit nicht möglich (Detailseite zeigt Metadaten/Status).
- **Mandantenfähigkeit:** Das Datenmodell führt `tenant_id`-Felder, die Anwendung erzwingt aber **keine** Mandanten-Trennung. Die Installation ist als Ein-Mandanten-Betrieb pro Behörde gedacht. Keine Mehr-Mandanten-Zusagen.

8. Administration

AKTUELL IMPLEMENTIERT (per Code und früherem Release-Commit verifiziert):

- Benutzerliste mit Rollen, Sperrstatus, letzter Anmeldung/Abmeldung (aus Audit), IP und Online-Status (aus SessionRegistry).
- **Sperren/Entsperren:** setzt das `locked`-Flag; gesperrte Benutzer können sich nicht mehr anmelden (per Test verifiziert) und werden sofort ausgeloggt (SessionRegistry-Expiry).
- **Löschen:** entfernt zuerst alle Refresh-Token-Sitzungen des Benutzers, beendet aktive Sitzungen, löscht dann den Account.
- **Selbstschutz:** Ein Administrator kann sich nicht selbst sperren oder löschen; der **letzte verbleibende Administrator** kann weder gesperrt noch gelöscht werden.
- Rollenänderungen werden als `ROLE_CHANGED`-Audit protokolliert; alle Verwaltungsaktionen landen im Audit.

Warum diese Schutzmechanismen existieren: Ein versehentliches Aussperren aller Administratoren wäre nur noch per Datenbankzugriff reparierbar. Die Schutzregeln verhindern genau diesen Zustand.

9. Audit und Nachvollziehbarkeit

AKTUELL IMPLEMENTIERT: Ereignisse werden in der Tabelle `audit_events` (PostgreSQL) gespeichert: Zeitstempel, Akteur, Tenant, Ereignistyp, Entität, Modul, Korrelations-ID, Request-Pfad, HTTP-Methode und **IP-Adresse**. Die Web-UI (Auditor/Admin) bietet Filterung, Detailansicht und CSV-Export.

Ereignis	Protokolliert	IP erfasst
Anmeldung (USER_LOGIN)	✓ (per Test verifiziert)	✓ (per Test verifiziert)
Fehlgeschlagene Anmeldung (USER_LOGIN_FAILED)	✗ — siehe Lücke unten	—
Abmeldung (USER_LOGOUT)	✓ (per Test verifiziert)	✓
Benutzer angelegt (USER_CREATED)	✓	—
Rollenänderung (ROLE_CHANGED)	✓	—
Sperren/Entsperren/Löschen	✓	✓ (über Request-Kontext)
Dokument-Upload/Ingestion	✓	✓
Such-/Abruf-Ausführung (RETRIEVAL_EXECUTED)	✓	✓
KI-Ausführung (MODEL_INFERENCE) inkl. Strategie	✓	✓
Token-Refresh/-Widerruf	✓	✓

Behoben (AKTUELL IMPLEMENTIERT, per Test verifiziert): Audit-Emissionen laufen seit dem Härtings-Update in einer **eigenen Transaktion** (`REQUIRES_NEW`) — `USER_LOGIN_FAILED` übersteht den Rollback der fehlgeschlagenen Anmeldung und wird **inkl. IP-Adresse** persistiert. Der Test `wrongPassword_failsAndFailedLoginAuditPersistsWithIp` verifiziert dieses Verhalten.

Wert für die Praxis: Anmeldungen (auch Fehlversuche mit Sperr-Grund), Abmeldungen und Verwaltungsaktionen sind lückenlos nachvollziehbar (wer, wann, von welcher IP) — Grundlage für Brute-Force-Erkennung und Incident-Analyse. KI-Ausführungen werden mit Strategie und Konfidenz protokolliert; die vollständigen Verifikationsschritte einer Antwort liegen derzeit nur im Server-Log, nicht im Audit-Speicher (Weiterentwicklung).

10. Browser- und API-Sicherheit

CSRF (AKTUELL IMPLEMENTIERT, per Test verifiziert): CSRF-Schutz aktiv; Token als `XSRF-TOKEN`-Cookie (bewusst nicht `HttpOnly`, damit `htmx` den `X-XSRF-TOKEN`-Header setzen kann). POST ohne Token → 403. Seit dem Härtungs-Update trägt das Cookie **SameSite=Lax** (per Test verifiziert); das `Secure`-Attribut folgt der Deployment-Konfiguration (`COOKIE_SECURE`, Standard: aus für die lokale HTTP-Demo, **an** hinter dem HTTPS-Reverse-Proxy).

CORS: Es gibt **keine** CORS-Konfiguration — Cross-Origin-Zugriffe aus Browsern werden nicht freigegeben; die Anwendung ist rein same-origin. Das ist für das aktuelle Modell korrekt und kein Handlungsbedarf.

HTTP-Security-Header (Web-App):

Header	Status
X-Frame-Options	deny ✓ (Clickjacking-Schutz)
X-XSS-Protection	deaktiviert (veraltet; Empfehlung P3: CSP stattdessen)
X-Content-Type-Options	nosniff aktiv (seit Härtungs-Update, per Test verifiziert)
Content-Security-Policy	nicht gesetzt in der Web-App (nur in der separaten SPA-App) → Empfehlung P3

API-Zugriff: Die Plattform-REST-API (`/api/**`, separate App) ist zustandslos mit JWT abgesichert; öffentlich sind nur `register`, `login`, `refresh`, `providers` — **Register ist kritisch** (siehe Abschnitt 5/20). In der Verwaltungsassistent-Installation existieren keine `/api/**`-Endpunkte.

11. Dokumentensicherheit und Verschlüsselung at Rest

Wo liegen welche Daten? (AKTUELL IMPLEMENTIERT)

Daten	Speicherort
Original-Datei (PDF/DOCX/...)	lokales Dateisystem (<code>uploads/</code> , Dateiname mit UUID-Präfix)
SHA-256-Prüfsumme + Metadaten	PostgreSQL (<code>documents</code> , <code>document_versions</code>)
Extrahierter Text (Chunks)	PostgreSQL (<code>search_document_chunks</code>)
Embeddings (Vektoren)	Qdrant (<code>mda_chunks</code> , 768-d)
Wissensgraph	Neo4j
Audit-Daten	PostgreSQL (<code>audit_events</code>)
Benutzer/Passwort-Hashes	PostgreSQL (<code>auth_users</code>)

Werden Dokumente verschlüsselt gespeichert? — Nein.

- **Dateisystem:** Rohdateien ohne Anwendungs-Verschlüsselung (kein AES, kein KMS).
- **Datenbanken:** keine Anwendungs-Verschlüsselung; keine TDE-Konfiguration vorgegeben.
- **Backups:** Es existiert keine eingebaute Backup-Verschlüsselung.

Klassifizierung: **Verschlüsselung at rest** = **X NICHT IMPLEMENTIERT** (Anwendungsebene);
infrastrukturseitige Verschlüsselung (Festplatte/LVM/Docker-Volumes) ist **DEPLOYMENT-ABHÄNGIG** und
Aufgabe des Betreibers. Dateisystem-Rechte (nur App-Benutzer darf `uploads/` lesen) sind Zugriffskontrolle, keine
Verschlüsselung.

12. Soll Verschlüsselung at Rest eingeführt werden?

Zwei Optionen:

Option A — Infrastruktur-/Speicherverschlüsselung (empfohlen als Basis): Vollverschlüsselung der Datenträger (z. B. LUKS/dm-crypt), verschlüsselte Docker-Volumes, verschlüsselte VM-Disks. Vorteile: einfach, transparent für die Anwendung, kein Schlüsselmanagement in der App, schützt gegen physischen Diebstahl von Servern/Platten. Nachteile: schützt nicht gegen logischen Zugriff auf dem laufenden System (z. B. kompromittierten App-Server) und nicht gegen unverschlüsselte Backups.

Option B — Anwendungsverschlüsselung (AES-256-GCM, pro Dokument/Schlüssel-Hierarchie): Verschlüsselt Inhalte, bevor sie Platte/Backup erreichen; schützt auch gegen logische Zugriffe außerhalb der App und gegen gestohlene Backups. Nachteile: Schlüsselmanagement (Master Key, Rotation), Such-/Index-Problematik (verschlüsselte Chunks können nicht per Klartext-Suche gefunden werden — das würde die Retrieval-Architektur betreffen), höhere Komplexität.

Pragmatische Empfehlung für eine Kommune:

1. **Priorität 1 (Betrieb):** Option A sofort — verschlüsselte Datenträger/Volumes **und verschlüsselte Backups** (ein verschlüsselter Primärspeicher mit unverschlüsseltem Backup ist kein vollständiges Modell).
2. **Option B nur, wenn** die Dokumente einen hohen Schutzbedarf haben (z. B. VS-Einstufung, besondere Sozialdaten) und das Schlüsselmanagement organisatorisch beherrscht wird. Für den Standard-Betrieb mit Option A ist der Zusatznutzen begrenzt, der Aufwand (Indexierung, Rotation) aber real.

Backups: Egal welche Option — Backups müssen verschlüsselt und zugriffsbeschränkt aufbewahrt werden, sonst ist das Sicherheitsmodell unvollständig. Ein gestohlenes Backup enthält sämtliche Dokumente, Chunks, Vektoren und Audit-Daten.

13. AI-/LLM-Datenschutz

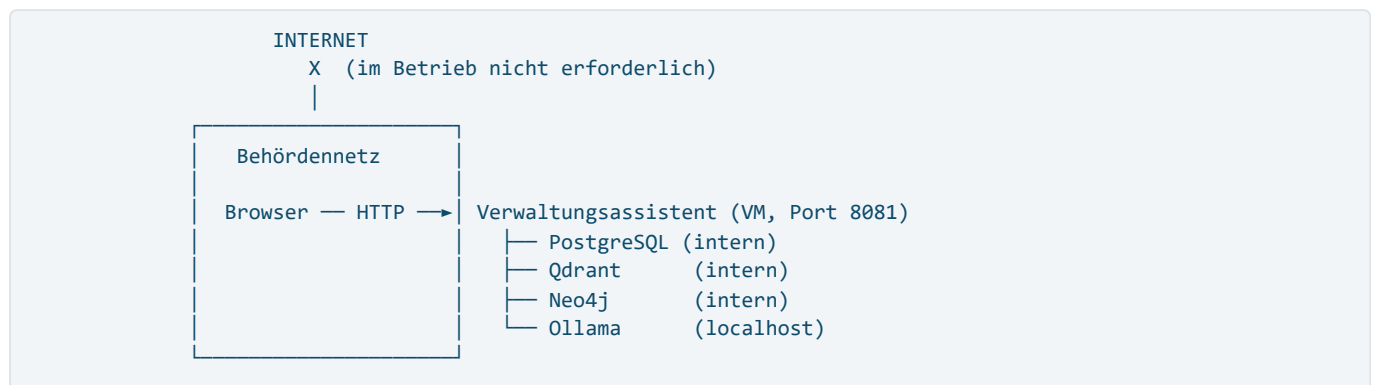
AKTUELL IMPLEMENTIERT:

- Modelle (Generation, Verifikation, Embeddings) laufen **lokal über Ollama** (localhost). Fragen, Dokumentauszüge und Antworten verlassen den Server nicht.
- Die Pipeline übergibt dem Generierungsmodell **nur die ausgewählten Belege** (max. 4 Dokumente, kurze Auszüge) — nicht den gesamten Korpus.
- Die **Verifikation** läuft als getrennter Schritt mit eigenem Modell; nicht belegte Antworten werden verworfen (Fail-Closed: „Für diese Frage liegen in der Wissensbasis keine ausreichenden Informationen vor.“).
- Ein optionales OpenAI-kompatibles Provider-Modul existiert im Code, ist aber **nicht aktiv**. Falls ein Kunde externe Modelle wünscht, wäre das eine bewusste Entscheidung mit Datenabfluss — dann greift dieses Kapitel nicht mehr.

Ehrliche Einordnung: Das ist keine „AI-Security“-Zusage gegen Prompt-Injection. Dokumente werden als unzuverlässiger Inhalt behandelt (Prompt-Injection über ein hochgeladenes Dokument ist denkbar; die Verifikations-/Fail-Closed-Mechanik begrenzt die Folgen, verhindert sie aber nicht). Wer Dokumente hochladen darf, ist über die Rechte (Korpus: Auditor/Admin) begrenzt. Empfehlung P3: explizite Prompt-Injection-Tests je Modell.

14. Air-Gap / Offline-Betrieb

AKTUELL IMPLEMENTIERT (Demo + VM-Bundle):



- **Laufzeit:** vollständig offline-fähig. Keine Lizenz-Dienste, keine Telemetrie, keine externen API-Aufrufe im Code-Pfad (einziger HTTP-Client: der lokale Modellserver; optionales, nicht aktives OpenAI-Modul).
- **Einmalig benötigt:** Modell-Downloads (Ollama), Container-Images (Docker), OS-/Java-Updates. Diese können auf einem vergleichbaren System vorbereitet und übertragen werden — dann ist auch die Installation offline möglich.
- **„100 % offline“ gilt für den Betrieb, nicht für die Einrichtung.** Software-Updates müssen über einen geregelten Prozess eingespielt werden (Empfehlung P2: definierter Update-/Medientransfer-Prozess).

15. Datenflüsse

Daten	Weg	Klassifizierung
Login-Daten (E-Mail + Passwort)	Browser → Verwaltungsassistent (HTTP; Produktion: HTTPS)	nie im Klartext gespeichert (BCrypt); Übertragung in Produktion TLS-pflichtig
Session-Cookie / CSRF-Token	Browser ↔ Verwaltungsassistent	Cookie; Produktion: Secure-Flag
Benutzerfrage	Browser → Verwaltungsassistent → (bei Abruf) lokale Suche + lokales LLM	bleibt auf dem Server
Hochgeladenes Dokument	Browser → Verwaltungsassistent → Dateisystem; Text → PostgreSQL; Vektoren → Qdrant; Graph → Neo4j	bleibt auf dem Server; unverschlüsselt at rest
LLM-Prompt / -Antwort	Verwaltungsassistent ↔ lokales Ollama	localhost, kein Netzaustritt
Audit-Ereignis	Verwaltungsassistent → PostgreSQL	bleibt auf dem Server

Kein Datenpfad verlässt den Server. Der einzige externe Pfad ist der Browser ↔ Verwaltungsassistent-Kanal, der in Produktion über HTTPS laufen muss.

16. Bedrohungsmodell

#	Bedrohung	Was heute schützt	Was fehlt	Empfehlung
1	Unauthentifizierter externer Angreifer	alles außer Login/Statik erfordert Anmeldung (verifiziert); keine CORS-Freigaben; CSRF aktiv	HTTP im Demo; keine Ratenbegrenzung	P1 HTTPS; P2 Rate-Limiting
2	Gestohlenes Passwort	BCrypt(12); Sperre durch Admin möglich; Audit zeigt Anmeldungen	kein MFA, kein automatisches Lockout nach Fehlversuchen	P3 MFA für Admins; P2 Brute-Force-Schutz
3	Gestohlenes Session-Cookie	serverseitige Sessions; Logout invalidiert; Admin kann Sitzungen beenden	Cookie ohne Secure-Flag im Demo	P2 Cookie-Flags; P1 HTTPS
4	Bösartiger angemeldeter Benutzer	Rollentrennung; Audit	Fall-Detail ohne Eigentümer-Prüfung (UUID reicht); Dokumentliste global	P2 Objekt-Prüfungen
5	Kompromittierter Admin-Account	Audit aller Aktionen; Selbst-/Letzter-Admin-Schutz verhindert Aussperren, nicht Missbrauch	kein MFA, keine Vier-Augen-Prinzipien	P3 MFA/Admin-Härtung
6	Direkter Datenbankzugriff	interne Ports nicht exponiert (VM-Deployment); DB-Passwörter per Umgebungsvariable	Standard-Passwörter im Demo-Compose	P1 echte Secrets; Netzsegmentierung
7	Gestohlenes Backup	— (keine Verschlüsselung vorgegeben)	Backup-Verschlüsselung	P1 verschlüsselte Backups
8	Kompromittierter App-Server	Rollen/Audit begrenzen den Schaden; Fail-Closed der KI	keine at-rest-Verschlüsselung	P1/P2 Verschlüsselungskonzept
9	Böswillig hochgeladenes Dokument (inkl. Prompt-Injection)	Upload nur angemeldet; Korpus-Verwaltung nur Auditor/Admin; Verifikation begrenzt unbelegte Antworten	keine dedizierten Injection-Tests	P3 Prompt-Injection-Tests
10	Prompt-Injection im Dokument	wie #9	wie #9	wie #9
11	Netzwerk-Abhören (Browser ↔ Server)	im Demo nichts (HTTP!)	TLS	P1 HTTPS via Reverse Proxy
12	Physischer Diebstahl von Server/Platte	— (keine Verschlüsselung)	Festplattenverschlüsselung	P1 Option A (Abschnitt 12)

17. Produktionsdeployment (Empfehlung für eine Kommune)

Verwaltungsassistent-Verantwortung (Anwendung):

- Anwendung als Container/Systemd-Dienst; Profile für Produktion; DB-Schema-Verwaltung.
- Rollenmodell, CSRF, Session-Handling, Audit (wie beschrieben).
- Korrekte Secrets-Behandlung: `JWT_SECRET`-Pflichtwert (kein Dev-Default), DB-/Neo4j-Passwörter aus Umgebung/Secret-Store.

Kunden-/Infrastrukturverantwortung (Betrieb):

Maßnahme	Priorität
Dedizierter Server/VM, nur Web-Port + SSH exponiert	P1
HTTPS-Reverse-Proxy mit gültigen Zertifikaten (+ HSTS)	P1
Firewall: 5432/6333/7687/11434 nur serverintern	P1
Verschlüsselte Datenträger/Volumes und verschlüsselte Backups	P1
OS-/Container-Security-Updates, geregelter Update-Prozess (auch offline)	P1
Echte, einzigartige Passwörter für PostgreSQL/Neo4j/Admin-Accounts; Demo-Accounts entfernen	P1
Audit-Aufbewahrung (Retention) und -Auswertung definieren	P2
Monitoring (Erreichbarkeit, Platten, Logs)	P2
Netzsegmentierung (Server in eigenem VLAN)	P2
MFA für Administratoren (z. B. über Proxy)	P3
Vier-Augen-Prinzip für kritische Admin-Aktionen	P3

18. Verantwortungsgrenzen — was Verwaltungsassistent nicht leistet

Verwaltungsassistent ist kein Komplett-Sicherheitsprodukt für die gesamte IT-Umgebung. Ausdrücklich **nicht** abgedeckt:

- Schutz eines bereits kompromittierten Betriebssystems oder Servers.
- Schutz vor einem gestohlenen Administrator-Passwort (ohne MFA).
- Netzwerk-Firewalling und Netzsegmentierung (Deployment-Aufgabe).
- Physische Sicherheit von Server und Speichermedien.
- Backup-Sicherheit (Verschlüsselung, Aufbewahrungsort).
- TLS-Zertifikatsverwaltung und -Erneuerung.
- OS-/Container-Härtung und Patch-Management.
- Rechtliche Bewertung (DSGVO-DPIA, Weisungsgebundenheit) — Verwaltungsassistent liefert die technischen Eigenschaften (lokale Verarbeitung, Rollen, Audit), keine Rechtsberatung.

Diese Abgrenzung ist bewusst: Sie verhindert, dass Sicherheitslücken der Umgebung der Anwendung zugeschrieben werden — und umgekehrt.

19. Sicherheitsstatus

Bereich	Status	Bemerkung
Passwort-Hashing	✓	BCrypt (Kostenfaktor 12), nie Klartext
Authentifizierung	✓	Form-Login, E-Mail + Passwort, Sperr-/Disabled-Prüfung
Fehlversuchs-Schutz	✓	zeitbasierte Sperre: 5 Fehlversuche/10 min → 10 min Sperre, konfigurierbar; erfolgreiche Anmeldung setzt zurück
JWT/Token (API-App)	✓	HS256, 15 min, Claims sub/user_id/roles
Refresh-Tokens (API-App)	✓	nur als SHA-256-Hash gespeichert, Rotation, Widerruf
Sessions	✓	serverseitig, 30 min, Registry, max. 10, Expiry; Cookie HttpOnly/SameSite=Lax
Logout	✓	Session invalidiert, Audit (verifiziert)
Rollen/Rechte	✓	Filterkette: ADMIN/ANALYST/USER/AUDITOR (verifiziert)
Objekt-Eigentümerschaft (Fälle)	✓	zentraler Zugriffswächter für Detail + alle Unteraktionen (403 bei fremden Fällen, verifiziert)
CSRF	✓	Token-Cookie mit SameSite=Lax; POST ohne Token = 403 (verifiziert)
CORS	✓	keine Freigaben (same-origin)
Security-Header	⦿	X-Frame-Options ✓; X-Content-Type-Options nosniff ✓; CSP offen → P3
Audit	✓	Login (inkl. Fehlversuche + Sperr-Grund), Logout, Admin, KI protokolliert inkl. IP; Emission in eigener Transaktion
IP-Erfassung	✓	bei Login/Logout/Request-Kontext (verifiziert)
Dokumentenspeicherung	✓	Dateisystem + PostgreSQL + Qdrant + Neo4j, Prüfsummen
Verschlüsselung at rest	✗	nicht implementiert; Infrastruktur-Verschlüsselung = Deployment-Aufgabe (P1)
Backup-Verschlüsselung	✗	nicht vorgegeben → P1 (Betrieb)
TLS	✗ (Demo)	HTTP; Produktion: HTTPS-Reverse-Proxy (P1)
Air-Gap-Betrieb	✓	Laufzeit offline; Einrichtung braucht Downloads
Lokales LLM	✓	Ollama lokal; kein Datenabfluss; externes Provider-Modul vorhanden, nicht aktiv
Mandanten-Trennung	✗	Ein-Mandanten-Modell; tenant_id-Felder vorhanden, nicht erzwungen

20. Empfehlungen (priorisiert)

Priorität 1 — vor Produktivbetrieb

1. **HTTPS** über Reverse Proxy; Session-/CSRF-Cookie-Flags (Secure, SameSite=Lax).
2. **Secrets:** `JWT_SECRET` setzen (kein Dev-Default), einzigartige DB-/Neo4j-Passwörter, Demo-Accounts entfernen.
3. **Verschlüsselung at rest (Option A)** inkl. **verschlüsselter Backups** — Umsetzung beim Betrieb, nicht in der App. (Begründung: physischer Diebstahl und Backup-Verlust sind die realistischsten Szenarien für eine Behörde; Infrastruktur-Verschlüsselung deckt sie ohne Eingriff in die Such-Architektur ab.)
4. **Registrierung/Rollen der Plattform-API einschränken**, falls diese App eingesetzt wird (öffentlicher Register-Endpunkt übernimmt Client-Rollen — kritisch).
5. OS-/Container-Update-Prozess, Firewall-Konfiguration gemäß Abschnitt 17.

Priorität 2 — dringend empfohlen

6. **Passwort-Zurücksetzung** einführen (gefährdeten/vergessenen Passwörtern geregelt begegnen).
7. Audit-Retention und Auswertung (Aufbewahrungsdauer, Alarme bei Sperr-/Lösch-Aktionen).
8. Vollständige Verifikations-Nachvollziehbarkeit im Audit (derzeit nur im Server-Log).

Priorität 3 — optional / Härtung

11. MFA für Administratoren (typischerweise am Proxy/IdP).
 12. CSP für die Web-App; Prompt-Injection-Tests je Modell.
 13. Secrets-Management (Vault o. ä.), Schlüsselrotation.
 14. SIEM-Anbindung des Audit-Exports; Vier-Augen-Prinzip für kritische Admin-Aktionen.
-

21. FAQ

Wer kann die Dokumente lesen? Alle angemeldeten Benutzer sehen die Dokumentliste (Ein-Organisations-Modell); die Originaldatei ist über die Web-UI derzeit nicht abrufbar (nur Metadaten/Status). Empfehlung P2: feinere Sichtbarkeitsregeln, falls gewünscht.

Werden unsere Dokumente an ChatGPT geschickt? Nein. Alle Modelle laufen lokal. Ein optionales Provider-Modul für externe APIs existiert, ist aber nicht aktiv.

Kann Verwaltungsassistent ohne Internet funktionieren? Ja, im Betrieb vollständig. Internet wird nur für die Erst-Einrichtung (Modell-Downloads, Images) und Updates benötigt.

Was passiert bei einem gestohlenen Passwort? Der Angreifer kann sich anmelden, solange das Konto aktiv ist. Alle Anmeldungen werden mit IP protokolliert; der Administrator kann das Konto sofort sperren (Logout erzwungen). Wiederholte Fehlversuche Dritter greifen in die zeitbasierte Sperre (5 Versuche/10 Minuten → 10 Minuten Sperre). Eine Passwort-Zurücksetzung existiert derzeit nicht (P2-Weiterentwicklung); MFA ist eine Empfehlung (P3).

Was passiert beim Logout? Die Sitzung wird serverseitig beendet, das Cookie gelöscht und der Logout protokolliert (verifiziert).

Kann ein Administrator Benutzer sperren? Ja — inklusive Sofort-Logout. Selbst-Sperren und Sperren des letzten Administrators sind verhindert (verifiziert).

Wo werden die Dokumente gespeichert? Lokales Dateisystem (Originale), PostgreSQL (Text/Metadaten), Qdrant (Vektoren), Neo4j (Graph) — alles auf dem Verwaltungsassistent-Server.

Sind die Dokumente verschlüsselt? Nein, aktuell nicht (weder Dateisystem noch Datenbank). Empfohlen: verschlüsselte Datenträger/Volumes und verschlüsselte Backups (P1, Betriebsaufgabe).

Was passiert mit Backups? Es gibt keine eingebaute Backup-Lösung; der Betrieb ist für Backups (und deren Verschlüsselung) verantwortlich.

Welche Daten werden protokolliert? Anmeldungen/Abmeldungen (mit IP), Benutzerverwaltung, Dokument- und Suchaktionen, KI-Ausführungen. Keine Passwörter, keine vollständigen Antwortinhalte in Metadaten außerhalb der fachlichen Daten.

Kann ein Benutzer die Daten eines anderen Benutzers sehen? Fälle: nein — Liste und Detailansicht (inkl. aller Unteraktionen) prüfen die Eigentümerschaft; fremde Fall-IDs werden mit 403 abgewiesen (verifiziert). Dokumente: für alle angemeldeten Benutzer sichtbar (Ein-Organisations-Modell).

Wo findet die KI-Verarbeitung statt? Vollständig auf dem Verwaltungsassistent-Server (lokales Ollama).

Kann die Anwendung in unserem Behördennetz betrieben werden? Ja — genau dafür ist sie gebaut: lokale Installation, offline-fähig, rollenbasierter Zugriff, Audit. TLS und Netzsegmentierung setzt der Betrieb um.

Anhang A — TLS-Konfiguration (Reverse Proxy, Beispiel)

EMPFOHLENE PRODUKTIONSKONFIGURATION — Beispiel für nginx (sinngemäß auf Caddy/Traefik übertragbar):

```
server {
    listen 443 ssl;
    server_name verwaltungsassistent.behoerde.de;

    ssl_certificate      /etc/ssl/verwaltungsassistent/fullchain.pem; # vom Betrieb gepflegt
    ssl_certificate_key  /etc/ssl/verwaltungsassistent/privkey.pem;

    # Härtung (Betriebsentscheidung)
    add_header Strict-Transport-Security "max-age=31536000" always;

    location / {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Voraussetzungen auf Anwendungsseite: `COOKIE_SECURE=true` (Secure-Flags für Session- und CSRF-Cookie), `JWT_SECRET` gesetzt, Demo-Accounts entfernt. Zertifikate, Erneuerung und HSTS bleiben Aufgabe des Betriebs — die Anwendung selbst terminiert kein TLS (bewusste Architekturentscheidung zugunsten des Reverse-Proxy-Modells).

Dieses Dokument basiert auf dem Stand vom 16.08.2026. Verifikation: 15 fokussierte Sicherheitstests (alle bestanden, eine Lücke dokumentiert), 171 Tests der schnellen Suite, Live-Prüfungen gegen die laufende Demo-Installation.